

Safe, Task-Consistent Manipulation with Operational Space Control Barrier Functions

Daniel Morton and Marco Pavone

Abstract—Safe real-time control of robotic manipulators in unstructured environments requires handling numerous safety constraints without compromising task performance. Traditional approaches, such as artificial potential fields (APFs), suffer from local minima, oscillations, and limited scalability, while model predictive control (MPC) can be computationally expensive. Control barrier functions (CBFs) offer a promising alternative due to their high level of robustness and low computational cost, but these safety filters must be carefully designed to avoid significant reductions in the overall performance of the manipulator. In this work, we introduce an Operational Space Control Barrier Function (OSCBF) framework that integrates safety constraints while preserving *task-consistent* behavior. Our approach scales to hundreds of simultaneous constraints while retaining real-time control rates, ensuring collision avoidance, singularity prevention, and workspace containment even in highly cluttered settings or during dynamic motions. By explicitly accounting for the task hierarchy in the CBF objective, we prevent degraded performance across both joint-space and operational-space tasks, when at the limit of safety. We validate performance in both simulation and hardware, and release our open-source high-performance code and media on our project webpage, <https://stanfordasl.github.io/oscbf/>

I. INTRODUCTION

With autonomous robotic manipulators increasingly operating in unstructured environments, real-time safe control is critical – not only for collision avoidance but also for a broad range of other safety constraints that are common to manipulators, such as singularity avoidance. An ideal control framework should enforce these safety constraints while minimally modifying the robot’s desired behavior, even in dynamic scenarios or when multiple constraints must be satisfied simultaneously.

Enforcing safety while retaining performance is particularly relevant for recent learning-based controllers, which may be quite versatile, but do not provide guarantees on safety [1]. Often, these learning-based policies operate in operational space, via end-effector motion commands. Similarly, when collecting data for imitation learning, teleoperated demonstrations often only command a motion of the end-effector [2, 3]. In both settings (training and deployment), the manipulator must rely on a lower-level operational space controller (OSC) [4] to map the end-effector motions back to the whole-body control of the robot. This introduces potential safety concerns at multiple levels: the operational space command itself may be unsafe, or the mapping back to

the whole-body motion may be unsafe. To maintain safety, the low-level controller must therefore consider multiple constraints, in both the operational and joint spaces.

Artificial potential fields (APFs) [5] have long served as a simple way of encoding safety in operational space control via repulsive forces around obstacles. However, APFs suffer from a few key problems: they can lead to oscillations when close to obstacles or moving quickly, they can influence the dynamics even far from an unsafe region, and they can require extensive tuning to avoid creating local minima [6, 7]. APFs can be designed to accommodate multiple task objectives and safety constraints [8] but this requires an explicit hierarchy and null space for each constraint, which does not scale to highly-constrained settings.

Model predictive control (MPC), alternatively, handles safety via constrained receding-horizon optimal control. However, MPC can face computational challenges, particularly if the system is subject to nonconvex constraints and nonlinear dynamics. For MPC to run at real-time rates on a manipulator, either safety conditions are not considered [9], or assumptions must be made about the reference trajectory [10].

Another approach, control barrier functions (CBFs) are a minimally-invasive means of enforcing safety for nonlinear systems via optimization-based controllers, and improve upon APFs with less computational demands than MPC [7]. Due to their robustness and real-time capabilities, CBFs have been successfully demonstrated on a wide range of systems, including manipulators [11]–[17].

Previous work on CBFs for manipulators often assume a reduced-order model with direct control over the joint velocities, and only extend safety conditions to the full-order dynamics with the assumption that a low-level controller can track a velocity command reasonably well [11, 15]. However, this condition does not hold under dynamic maneuvers, and cannot be used with compliant control. Often, these safety criteria are analyzed only over joint-space motions, neglecting the impact on the operational-space dynamics which are critical for teleoperation and learning-based control [17]. Works that analyze the operational space [12, 13] often neglect how to best balance hierarchical task performance and safety in the joint space and operational space, or handling multiple safety constraints.

A. Statement of Contributions

We present a framework for integrating Control Barrier Functions (CBFs) into Operational Space Control (OSC) which (1) enforces safety while respecting the hierarchical

Daniel Morton was supported by a NASA Space Technology Graduate Research Opportunity

Daniel Morton and Marco Pavone are with the Departments of Mechanical Engineering and Aeronautics & Astronautics, Stanford University, Stanford, CA 94305. {dmorton, pavone}@stanford.edu

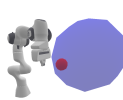
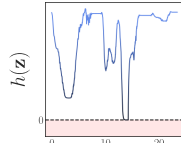
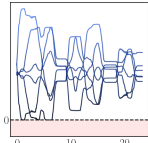
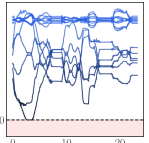
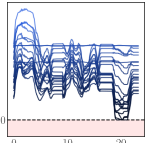
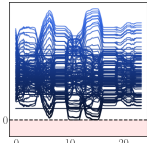
Safety Constraint	Singularity avoidance	End-effector containment	Joint limit avoidance	Whole-body collision avoidance	Whole-body containment
Visualization					
Number of CBFs	1	6	14	21	126
Constraint Evolution					

Fig. 1: **Safe and performant highly-constrained manipulation.** Our OSCBF controller maintains safety for hundreds of constraints enforced concurrently, using the full second-order robot dynamics and torque control, and operates at real-time control rates (over 1000 Hz). Shown above is the simultaneous constraint evolution for all CBFs during one adversarial trajectory from teleoperation (end-effector target shown in red). Across 168 safety conditions, the robot remains safe ($h(\mathbf{z}) > 0$) without over-conservative behavior near the boundary of safety ($h(\mathbf{z}) = 0$).

task definition of OSC across the operational and joint spaces, (2) applies to both kinematically- and dynamically-controlled manipulators, and (3) scales to large numbers of safety conditions while maintaining real-time control rates, even when performing dynamic motions through highly-occluded environments. Additionally, we release our software, CBFpy [18], which provides an easy-to-use and high-performance interface for CBF controller design. The code, along with additional media, is available at <https://stanfordasl.github.io/oscbf/>

B. Paper Organization

In Section II, we review the construction of control barrier functions for constraints of relative degree 1 and 2, and discuss practical application of CBFs for highly-constrained systems. In Section III, we review manipulator kinematics, dynamics, and tasks. In Section IV, we detail the necessity of *task-consistent* safety filters, then we construct our Operational Space Control Barrier Function (OSCBF) controller, for both velocity- and torque- controlled robots. In Section V, we present analysis of the OSCBF controller’s performance and computational efficiency for five common types of safety constraints, then, in simulation and on hardware, we evaluate the controller in highly constrained environments and during dynamic motions.

II. CONTROL BARRIER FUNCTIONS

Consider a continuous-time dynamical system in control-affine form:

$$\dot{\mathbf{z}} = \mathbf{f}(\mathbf{z}) + \mathbf{g}(\mathbf{z})\mathbf{u} \quad (1)$$

with state $\mathbf{z} \in \mathcal{Z} \subseteq \mathbb{R}^n$, input $\mathbf{u} \in \mathcal{U} \subseteq \mathbb{R}^m$, and locally Lipschitz continuous dynamics functions $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^n$ and $\mathbf{g} : \mathbb{R}^n \rightarrow \mathbb{R}^{n \times m}$.

Safety can be posed through the lens of set invariance. For a safe subset of the state space, $\mathcal{C} \subset \mathcal{Z}$, if we can define a control barrier function $h(\mathbf{z}) : \mathbb{R}^n \rightarrow \mathbb{R}$ where $\mathcal{C}$ is the zero-superlevel set of h , then a controller satisfying

$$\dot{h}(\mathbf{z}, \mathbf{u}) \geq -\alpha(h(\mathbf{z})) \quad (2)$$

for $\mathbf{u} \in \mathcal{U}$ and extended class $\mathcal{K}_\infty$ function α will render $\mathcal{C}$ forward-invariant [19].

This condition, Eq. 2, can be integrated into a quadratic program (QP) convex optimization problem, paired with a min-norm objective to operate as a safety filter on a nominal (unsafe) controller:

$$\begin{aligned} \underset{\mathbf{u}}{\text{minimize}} \quad & \|\mathbf{u} - \mathbf{u}_{\text{nom}}\|_2^2 \\ \text{subject to} \quad & L_f h(\mathbf{z}) + L_g h(\mathbf{z})\mathbf{u} \geq -\alpha(h(\mathbf{z})) \end{aligned} \quad (3)$$

where L_f and L_g are the Lie derivatives of h along the dynamics, and $\dot{h}(\mathbf{z}, \mathbf{u}) = L_f h(\mathbf{z}) + L_g h(\mathbf{z})\mathbf{u}$.

The *relative degree* of a CBF refers to the number of differentiations along the dynamics required before the control input $\mathbf{u}$ explicitly appears. CBFs require a relative degree of 1 (RD1), but with mechanical systems, CBFs are often of relative degree 2 (RD2) [16]. This high relative degree implies that $L_g h(\mathbf{z}) = 0$, and thus, we must differentiate along the dynamics again. The second time-derivative of $h(\mathbf{z})$ can be expressed as

$$\ddot{h}(\mathbf{z}, \mathbf{u}) = L_f^2 h(\mathbf{z}) + L_g L_f h(\mathbf{z})\mathbf{u} \quad (4)$$

where $L_g L_f h(\mathbf{z}) \neq 0$ for a RD2 CBF.

Given this, we can construct a High-Order CBF (HOCBF) [20] for these RD2 constraints. Let $h_2(\mathbf{z}) = L_f h(\mathbf{z}) + \alpha(h(\mathbf{z}))$. Then, we modify the constraint in Eq. 3 to

$$L_f h_2(\mathbf{z}) + L_g h_2(\mathbf{z})\mathbf{u} \geq -\alpha_2(h_2(\mathbf{z})) \quad (5)$$

for an additional class $\mathcal{K}_\infty$ function α_2

An overview of the relative degree of various safety conditions for manipulators can be found in Table I.

Remark: When enforcing a large number of CBFs, particularly with input constraints, these will sometimes be in conflict, resulting in an infeasible QP. Finding valid, non-overly-conservative CBFs is an ongoing research challenge [21], but in practice, relaxing the QP results in a reasonable solution that enforces (but does not guarantee) safety in most cases.

TABLE I: SAFETY CONDITIONS: QP CONSTRAINT OR CBF

Safety Condition	Velocity Control	Torque Control
Joint position limit	RD1 CBF	RD2 CBF
Joint velocity limit	Constraint	RD1 CBF
Joint torque limit	—	Constraint
Operational position limit	RD1 CBF	RD2 CBF
Operational velocity limit	Constraint	RD1 CBF
Operational wrench limit	—	Constraint
Singularity avoidance	RD1 CBF	RD2 CBF
Collision avoidance	RD1 CBF	RD2 CBF

Given this, we can relax the QP (Eq. 3) by introducing a slack variable, $\mathbf{t}$ to handle the constraint violation with a large penalty, ρ :

$$\begin{aligned} \underset{\mathbf{u}}{\text{minimize}} \quad & \|\mathbf{u} - \mathbf{u}_{\text{nom}}\|_2^2 + \rho^T \mathbf{t} \\ \text{subject to} \quad & L_f h(\mathbf{z}) + L_g h(\mathbf{z}) \mathbf{u} \geq -\alpha(h(\mathbf{z})) - \mathbf{t} \\ & \mathbf{t} \geq \mathbf{0} \end{aligned} \quad (6)$$

III. MANIPULATOR CONTROL

A. Manipulator Kinematics

We consider a serial-chain manipulator with a kinematic model of the following form:

$$\boldsymbol{\nu}(\mathbf{q}, \dot{\mathbf{q}}) = J(\mathbf{q})\dot{\mathbf{q}} \quad (7)$$

where the Jacobian J maps joint velocities $\dot{\mathbf{q}} \in \mathbb{R}^{n_q}$ to the operational space twist $\boldsymbol{\nu} = [\dot{\mathbf{x}}_p, \boldsymbol{\omega}] \in \mathbb{R}^6$.

We denote a generalized inverse of the Jacobian J as $J^\#$, and for a non-redundant manipulator, $J^\# = J^{-1}$. Given this, the relationship between operational space twist and joint velocities can be expressed as:

$$\dot{\mathbf{q}} = J^\#(\mathbf{q})\boldsymbol{\nu} + N(\mathbf{q})\dot{\mathbf{q}}_0 \quad (8)$$

where $N(\mathbf{q})$ is the null space projection matrix associated with $J^\#$,

$$N(\mathbf{q}) = I - J^\#(\mathbf{q})J(\mathbf{q}) \quad (9)$$

and $\dot{\mathbf{q}}_0$ is an arbitrary joint velocity vector.

B. Manipulator Dynamics

We consider a serial-chain manipulator with a dynamics model of the following form:

$$M(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{g}(\mathbf{q}) = \boldsymbol{\Gamma} \quad (10)$$

where $M(\mathbf{q})$ is the mass matrix, $\mathbf{c}(\mathbf{q}, \dot{\mathbf{q}})$ is the vector of centrifugal and Coriolis forces, $\mathbf{g}(\mathbf{q})$ is the gravity vector, and $\boldsymbol{\Gamma}$ is the vector of joint torques.

Likewise, we can describe the dynamic behavior of the manipulator's end-effector in operational space as follows:

$$\Lambda(\mathbf{q})\dot{\boldsymbol{\nu}} + \boldsymbol{\mu}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{p}(\mathbf{q}) = \mathcal{F} \quad (11)$$

where $\Lambda(\mathbf{q})$ is the operational space mass matrix, $\boldsymbol{\mu}(\mathbf{q}, \dot{\mathbf{q}})$ is the operational space centrifugal and Coriolis force vector, $\mathbf{p}(\mathbf{q})$ is the operational space gravity vector, and $\mathcal{F}$ is the operational space wrench.

The components of the joint-space and operational-space dynamic models are related via the following:

$$\Lambda(\mathbf{q}) = (J(\mathbf{q})M^{-1}(\mathbf{q})J^T(\mathbf{q}))^{-1} \quad (12)$$

$$\boldsymbol{\mu}(\mathbf{q}, \dot{\mathbf{q}}) = \bar{J}^T(\mathbf{q})\mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) - \Lambda(\mathbf{q})\dot{J}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} \quad (13)$$

$$\mathbf{p}(\mathbf{q}) = \bar{J}^T(\mathbf{q})\mathbf{g}(\mathbf{q}) \quad (14)$$

where $\bar{J}(\mathbf{q})$ is the dynamically-consistent generalized inverse Jacobian for a redundant manipulator,

$$\bar{J}(\mathbf{q}) = M^{-1}(\mathbf{q})J^T(\mathbf{q})\Lambda(\mathbf{q}) \quad (15)$$

For a non-redundant manipulator, these same equations apply where $\bar{J}(\mathbf{q}) = J^{-T}(\mathbf{q})$

The relationship between operational space wrenches and joint space torques can be expressed as:

$$\boldsymbol{\Gamma} = J^T(\mathbf{q})\mathcal{F} + N^T(\mathbf{q})\boldsymbol{\Gamma}_0 \quad (16)$$

where $N^T(\mathbf{q})$ is the null space projection matrix associated with J^T ,

$$N^T(\mathbf{q}) = I - J^T(\mathbf{q})\bar{J}^T(\mathbf{q}) \quad (17)$$

I is an identity matrix, and $\boldsymbol{\Gamma}_0$ is an arbitrary joint torque vector which will be projected into the null space of $J^T(\mathbf{q})$. We denote a vector of joint torques in the null space as $\boldsymbol{\Gamma}_N$

C. Task Hierarchy

For our experiments, we consider the primary *task* as tracking a desired operational space pose $\mathbf{p} \in \mathbb{R}^6$, though other tasks can similarly apply (e.g., position tracking $\in \mathbb{R}^3$ using the linear Jacobian, J_v). For a redundant robot, a common secondary task is to maintain a joint posture with high manipulability characteristics, in which case $N(\mathbf{q})\dot{\mathbf{q}}_0$ (velocity control) or $N^T(\mathbf{q})\boldsymbol{\Gamma}_0$ (torque control) can be used to approach this posture without affecting the end-effector.

IV. OPERATIONAL SPACE CONTROL BARRIER FUNCTIONS

A. Task Consistency

When imposing a safety constraint on a robotic manipulator, many trajectories can maintain safety, but not all can additionally maximize task performance. As shown in Fig. 2, while “task-inconsistent” CBFs can remain safe (i.e., $h(\mathbf{z}) \geq 0$), they can also introduce excess motion at the boundary of safety. Examples of “task-inconsistency” include: (1) optimizing a joint-space metric, while the task is defined in the operational space; (2) the converse: optimizing an operational-space metric, while ignoring a secondary null space joint task; (3) optimizing a torque-based metric, rather than an acceleration-based metric.

In general, *task consistency* implies that rather than applying the CBF safety filter directly to the control input, the filter should minimally modify an output that reflects the task and hierarchy definition. By doing so, this eliminates unnecessary motion when the reference command moves further into the unsafe set. Inconsistent examples (1) and (2) above misrepresent the task hierarchy. (3) optimizes a metric which less directly reflects the position-based tasks than acceleration, due to the additional inertial mapping involved.

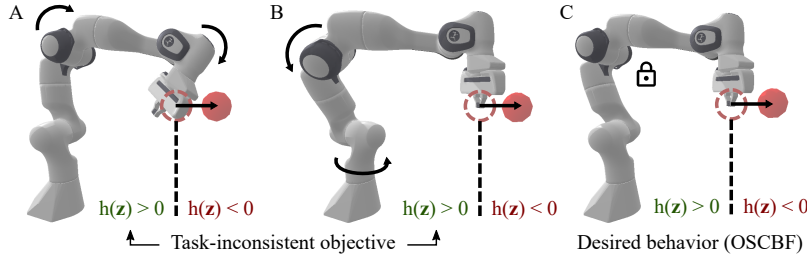


Fig. 2: **Task-consistency: balancing safety and performance.** Consider the behavior of the robot with the tip of the end-effector at the boundary of safety, where the desired goal moves towards the unsafe region. If the CBF objective is not *task-consistent*, the safety filter leads to a decrease in performance, even if safety is maintained. In (A), a joint space metric leads to a decrease in the operational space task performance, and in (B), a purely operational space metric leads to excess motion in the null space. (C) With OSCBF, excess motion is minimized and both safety and task performance is maintained.

B. Kinematic-OSCBF

For velocity-controlled manipulators, computing the safe control input $\dot{\mathbf{q}}$ requires three steps: (1) Determine the nominal velocities to reduce error in the task hierarchy; (2) Map the nominal velocities to an (unsafe) joint velocity command; (3) Pass the nominal joint velocities through the OSCBF QP to yield the safe velocity command $\dot{\mathbf{q}}^*$.

We begin by computing the nominal task velocities, $\boldsymbol{\nu}$ and $\dot{\mathbf{q}}_N$, where $\boldsymbol{\nu}$ is a proportional operational space twist command $\in \mathbb{R}^6$:

$$\boldsymbol{\nu} = \boldsymbol{\nu}_{\text{des}} - K_{po} \begin{bmatrix} \mathbf{x}_p - \mathbf{x}_{p,\text{des}} \\ \delta\phi \end{bmatrix} \quad (18)$$

and $\dot{\mathbf{q}}_N$ is a proportional joint velocity command $\in \mathbb{R}^{n_q}$, projected into the nullspace of the operational space task:

$$\dot{\mathbf{q}}_N = N(\mathbf{q}) (\dot{\mathbf{q}}_{\text{des}} - K_{pj}(\mathbf{q} - \mathbf{q}_{\text{des}})) \quad (19)$$

Here, $\delta\phi$ represents the instantaneous angular error vector between $\mathbf{x}_r$ and $\mathbf{x}_{r,\text{des}}$. We can easily reconstruct the rotation matrices R and R_{des} from the flattened representation $\mathbf{x}_r$ and compute this as follows, where $\mathbf{r}_i$ is the i^{th} column of R :

$$\delta\phi = -\frac{1}{2}(\mathbf{r}_1 \times \mathbf{r}_{1,\text{des}} + \mathbf{r}_2 \times \mathbf{r}_{2,\text{des}} + \mathbf{r}_3 \times \mathbf{r}_{3,\text{des}}) \quad (20)$$

K_{pj} and K_{po} are proportional gains for the joint space and operational space, respectively.

Combining these together, the nominal (unsafe) joint velocity command is

$$\dot{\mathbf{q}}_{\text{nom}} = J^\#(\mathbf{q})\boldsymbol{\nu} + \dot{\mathbf{q}}_N \quad (21)$$

To construct the CBF, we first define the control-affine joint-space dynamics ($\mathbf{z} = \mathbf{q} \in \mathbb{R}^n$), with respect to the joint velocity command ($\mathbf{u} = \dot{\mathbf{q}} \in \mathbb{R}^m$). For velocity control, we apply a reduced-order model where we assume direct control of the joint velocities.

$$\dot{\mathbf{z}} = \mathbf{u} \quad (22)$$

With these dynamics, and a CBF h , we can then construct the OSCBF QP:

$$\begin{aligned} &\underset{\dot{\mathbf{q}}}{\text{minimize}} \quad \|W_j(\dot{\mathbf{q}}_N - \dot{\mathbf{q}}_{N,\text{nom}})\|_2^2 + \|W_o(\boldsymbol{\nu} - \boldsymbol{\nu}_{\text{nom}})\|_2^2 \\ &\text{subject to} \quad L_f h(\mathbf{z}) + L_g h(\mathbf{z})\mathbf{u} \geq -\alpha(h(\mathbf{z})) \end{aligned} \quad (23)$$

Here, the CBF constraint is paired with a task-consistent objective which minimizes deviations from the nominal joint

and operational space velocities. W_j and W_o are positive-definite diagonal matrices, which can optionally be used to adjust the objective weighting between joint space and operational space deviations (for instance, to adjust the relative magnitudes between position and orientation components, or between prismatic and revolute joints).

Equivalently, the objective function of Eq. 23 can be written as

$$\|W_j N(\mathbf{q})(\dot{\mathbf{q}} - \dot{\mathbf{q}}_{\text{nom}})\|_2^2 + \|W_o J(\mathbf{q})(\dot{\mathbf{q}} - \dot{\mathbf{q}}_{\text{nom}})\|_2^2 \quad (24)$$

which we can then use to construct a QP with an objective of form $\frac{1}{2}\mathbf{x}^T P_{QP}\mathbf{x} + \mathbf{q}_{QP}^T \mathbf{x}$, where

$$P_{QP} = N^T W_j^T W_j N + J^T W_o^T W_o J \quad (25)$$

$$\mathbf{q}_{QP}^T = -\dot{\mathbf{q}}_{\text{nom}}^T P_{QP} \quad (26)$$

While we consider a hierarchy of only $n_t = 2$ tasks, this objective can also generalize to larger hierarchies, by considering the effect of the safety filter on each task i :

$$\sum_{i=1}^{n_t} \|W_i(\mathbf{v}_i(\mathbf{q}, \dot{\mathbf{q}}) - \mathbf{v}_{i,\text{nom}}(\mathbf{q}, \dot{\mathbf{q}}_{\text{nom}}))\|_2^2 \quad (27)$$

where $\mathbf{v}_i$ is a velocity, in operational or joint space.

Joint velocity limits can be encoded into the QP (Eq. 23) via an additional $2n_q$ constraints:

$$\dot{\mathbf{q}}_{\min} \leq \dot{\mathbf{q}} \leq \dot{\mathbf{q}}_{\max} \quad (28)$$

Remark: Adding input constraints to this QP does not guarantee forward invariance of the safe set, but it often still works in practice without introducing more complexity. For a more thorough handling, see input-constrained CBFs [22].

The optimizer of the QP (Eq. 23), $\dot{\mathbf{q}}^*$, can then be safely sent to the robot.

C. Dynamic-OSCBF

For torque-controlled manipulators, computing the safe control input $\boldsymbol{\Gamma}$ requires three steps: (1) Determine the nominal generalized forces to reduce error in the task hierarchy; (2) Map the nominal generalized forces to an (unsafe) joint torque command; (3) Pass the nominal joint torque command through the OSCBF QP to yield the safe torque command $\boldsymbol{\Gamma}^*$. A block diagram of this process can be found in Fig. 3.

TABLE II: OSCBF CONTROL FREQUENCIES (KHZ): 7-DOF FRANKA EMIKA PANDA

Experiment	CBF Constraints	Velocity Control		Torque Control	
		Mean	Lower bound (5th percentile)	Mean	Lower bound (5th percentile)
Singularity avoidance	1	9.55	5.15	6.14	3.89
End-effector safe-set containment	6	9.49	6.19	7.17	4.12
Joint limit avoidance	14	9.99	6.15	7.72	4.45
Whole-body collision avoidance	21	7.48	4.40	5.71	3.06
Whole-body safe-set containment	126	4.94	3.33	3.89	2.59
(All of the above)	168	3.24	2.35	2.94	2.25

dynamics are evaluated, and with over 100 CBF constraints. If C++ code is required, the OSCBF can be directly converted to a high-level optimized XLA representation and called via the XLA C++ interface. Compute times are from an Intel i7-1360p (5 GHz) NUC computer with 64 GB RAM, and Jax v.0.4.30. JIT compilation times are typically 2-5 seconds.

B. Multiple Safety Constraints

Manipulator control often requires handling multiple safety constraints simultaneously. For our experiments, we consider five common types of constraints: (1) singularity avoidance, (2) end-effector safe-set containment, (3) joint limit avoidance, (4) whole-body collision avoidance, and (5) whole-body safe-set containment.

To validate that safety is enforced for all constraints, we command an (unsafe) teleoperated end-effector trajectory that approaches the limit of safety for each constraint (Fig. 1). Each constraint is driven towards the boundary of safety ($h(\mathbf{z}) = 0$), yet none enter the unsafe region ($h(\mathbf{z}) < 0$), and no conditions restrict the robot motion prematurely, indicating excellent tracking performance, without over-conservatism in the face of safety. Additionally, even with 168 CBF constraints enforced concurrently, the OSCBF controller is *fast*: retaining over 1000 Hz and real-time control rates for velocity and torque control (Table II).

Below, we indicate the CBF construction for each constraint. Note: the following CBFs are of relative degree 1 for velocity control, and relative degree 2 for torque control. For torque control, we apply Eq. 5 to resolve the relative degree.

1) *Singularity avoidance*: As in [14], we define the barrier function based on the manipulability index $\mu(\mathbf{q})$ [26] and a tolerance ϵ ,

$$h(\mathbf{z}) = \mu(\mathbf{q}) - \epsilon \quad (43)$$

where $\mu(\mathbf{q})$ is equal to the product of the singular values σ of the end-effector Jacobian,

$$\mu(\mathbf{q}) = \prod_{i=1}^6 \sigma_i \quad (44)$$

2) *Joint position limit avoidance*: We construct the joint limit CBF as simply the joint-space distance to the minimum and maximum values,

$$h(\mathbf{z}) = \begin{bmatrix} \mathbf{q} - \mathbf{q}_{\min} \\ \mathbf{q}_{\max} - \mathbf{q} \end{bmatrix} \quad (45)$$

3) *Task position limit avoidance*: We assume an axis-aligned bounding box representation of the end-effector safe set, with a barrier function

$$h(\mathbf{z}) = \begin{bmatrix} \mathbf{x}_{p,EE}(\mathbf{q}) - \mathbf{x}_{p,\min} \\ \mathbf{x}_{p,\max} - \mathbf{x}_{p,EE}(\mathbf{q}) \end{bmatrix} \quad (46)$$

where $\mathbf{x}_{p,EE}(\mathbf{q})$ represents the position of the end-effector, after propagation through the forward kinematics.

Remark: Barrier functions can be constructed for a variety of keep-in (or keep-out) zones, not just axis-aligned boxes. Ellipsoids, halfspaces, polytopes, capsules, and more are all possible. For further discussion, refer to [27, 28].

4) *Whole-body collision avoidance*: We assume a sphere-decomposition collision model of the robot and the environment, with barrier functions

$$h_{ij}(\mathbf{z}) = \|\mathbf{x}_{p,j,\text{obs}} - \mathbf{x}_{p,i}(\mathbf{q})\|_2 - r_{j,\text{obs}} - r_i \quad (47)$$

for all i positions $\mathbf{x}_{p,i}(\mathbf{q})$ and radii r_i in the collision model of the robot, after propagation through the forward kinematics, and all j positions/radii of the environmental collision model, $\mathbf{x}_{p,j,\text{obs}}$ and $r_{j,\text{obs}}$. Our simple collision model for the Franka Panda consists of 21 spheres, and we emphasize that this will still work with a more detailed collision model.

5) *Whole-body set containment*: We assume an axis-aligned bounding box representation of the whole-body safe set, and again a sphere-based collision model of the robot.

$$h_i(\mathbf{z}) = \begin{bmatrix} \mathbf{x}_{p,i} - \mathbf{x}_{p,\min} - r_i \\ \mathbf{x}_{p,\max} - \mathbf{x}_{p,i} - r_i \end{bmatrix} \quad (48)$$

C. Highly-Constrained Environments

Operating in an unstructured environment such as the household often implies a lot of clutter: to name a few, reaching into narrow cabinets filled with items, moving dishes in and out of the sink, or cleaning up laundry [2]. To validate the performance of the controller in a similar setting, we consider a cluttered tabletop environment, where we enforce collision avoidance with both the table and a variable number of bodies (0 to 50), represented as a randomly generated set of spherical regions. The table is modeled with a halfspace CBF (as in Eq. 48, but only along the z direction), and each additional CBF constraint (Eq. 47) pairs one collision sphere on the robot with one collision sphere in the environment. Given the 21 spherical bodies in our simplified collision model of the Franka Panda, this implies that we have more than 1000 CBF constraints

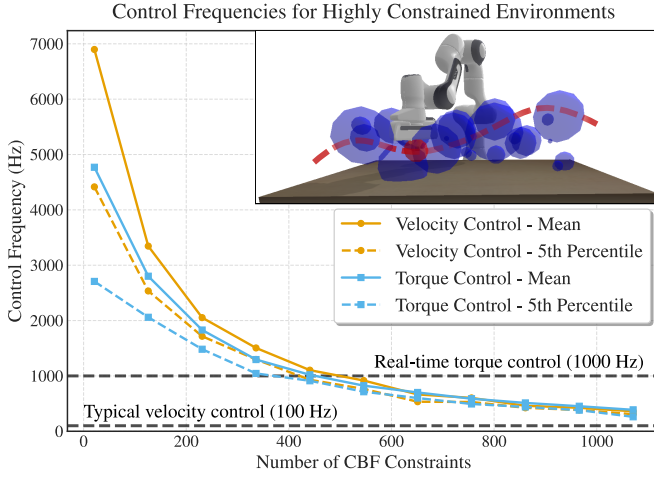


Fig. 4: **Scaling up collision avoidance.** Even in highly-cluttered scenes, our OSCBF controller maintains safety, task-tracking performance, and real-time control rates. Consider a tabletop environment (inset) with many randomly-generated collision bodies, shown in blue. OSCBF scales to *over 400 CBF constraints while retaining real-time control rates (1000 Hz) for torque control*, and well over 1000 constraints while retaining good control rates (100 Hz) for velocity control. We indicate both the mean frequency and the minimum frequency, assuming an allowable 5% packet drop.

when we enforce whole-body collision avoidance with 50 environmental bodies.

As shown in Fig. 4, we retain real-time control rates (1000 Hz) for over 400 CBF constraints, and maintain good control rates for velocity control (well over 100 Hz) even with over 1000 CBF constraints in the QP. We track an end-effector trajectory that safely moves through even the most tightly-occluded sections of the environment with no collisions – a task that would be infeasible for APFs due to the interference between repulsive potentials, and infeasible for MPC due to the high number of nonconvex constraints. Notably, we can ensure safety even with dynamic motions through the cluttered scene, given the real-time performance of torque control, with centrifugal and Coriolis compensation.

Remark: We can further increase computational efficiency by considering only the closest subset of collision pairs: this simple heuristic can dramatically raise the number of collision bodies in the environment for a given compute frequency. However, results show that even without these heuristics, we can enforce whole-body collision avoidance in high levels of clutter.

D. Dynamic Safety

Manipulators are often capable of high-speed dynamic motions, but guaranteeing safety during these motions can be difficult, particularly in an online setting without pre-planned trajectories. Previous works assumed that a low-level tracking controller can guarantee that a safe velocity command can always be met, but we emphasize that this is *not* the case for dynamic motions, with torque limits. As shown in Fig. 5, making this assumption can sometimes maintain safety (here, the end-effector trajectory does remain within the safe set), but the desired safe velocity reference cannot always be met, given the torque limits. This leads to

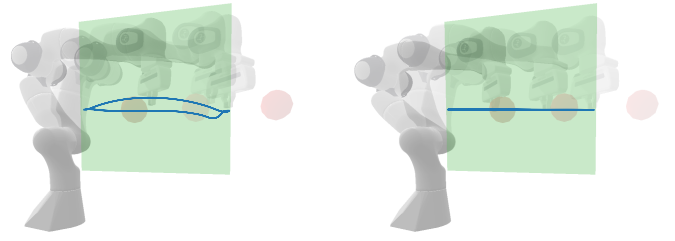


Fig. 5: **Dynamic task consistency under input constraints.** Under high-speed unsafe motions, accounting for the full dynamics of the robot and torque input constraints is necessary for good task performance. Consider a periodic, straight-line end-effector trajectory that commands a rapid motion of the end-effector tip into the unsafe set. With constraints on both the maximum joint velocities and torque, both a velocity-controlled robot (left) and a torque-controlled robot (right) maintain safety. However, the velocity CBF causes a degradation of tracking performance, due to instantaneously infeasible velocity commands, given the configuration of the robot and its torque limits.

a degradation in task tracking performance, as indicated by the departure from the desired straight-line motion. However, if the full second-order dynamics of the manipulator are considered, with explicit constraints on the torque input, we can maintain *both* tracking performance and safety.

E. Hardware Experiments

To validate OSCBF’s performance on hardware, we set up a 1 kHz real-time torque control interface for safe teleoperation and operational space control of the Franka Panda. Across four experiments designed to replicate the simulation results, OSCBF demonstrates remarkably smooth, low-latency tracking of the end-effector commands, safely approaching the boundary of safety within just 6 mm (Fig. 6 A/B/D) or quickly moving in and out of near-singular configurations ($\mu = 1e-2$) (Fig. 6 C) while operating at the dynamic limits of the Franka hardware. Fig. 6 B also demonstrates whole-body collision avoidance while maintaining good tracking of the operational space task, by filtering the secondary null space posture task when constrained by the environment. Notably, we observe effectively zero mismatch between the hardware data and simulation, save for a miniscule amount of latency (50 ms) from the teleoperation interface. Videos of the experiments can be found on our project webpage.

F. Discussion

1) *Parameter tuning:* For a good balance between performance and conservatism in safety constraints, the controller gains $K_{po}, K_{do}, K_{pj}, K_{dj}$, CBF class $\mathcal{K}_\infty$ functions α, α_2 (for each type of CBF), and objective weights W_j, W_o can be tuned. In general, the designer should (1) tune the controller gains to achieve good unconstrained performance, (2) enforce constraints and raise α, α_2 starting from 1 until velocity/torque limits are nearly reached under dynamic motions, and (3) adjust W_j, W_o starting from I to relax or strengthen deviations in end-effector position, rotation, or joint motions. *Remark:* OSCBF works even with minimal tuning. For all experiments, $\alpha = \alpha_2 = 10$ (for all CBFs) and $W_j = I_{n_q}, W_o = I_6$ performed well.

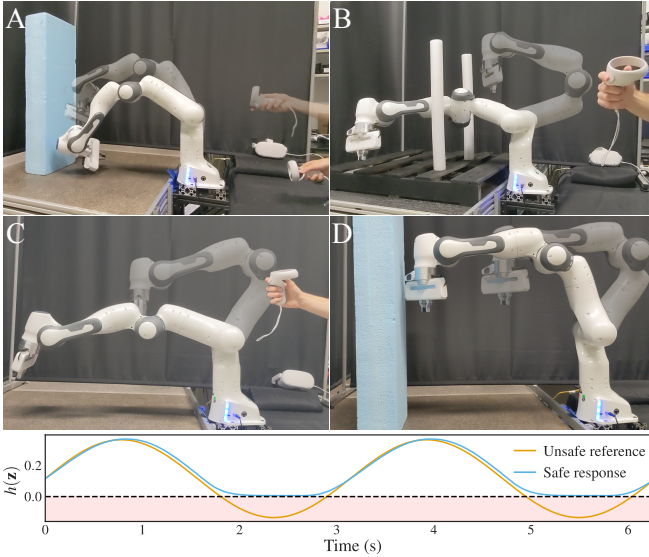


Fig. 6: **Hardware validation.** We evaluate OSCBF’s performance across four scenarios on the Franka Emika Panda hardware. In three teleoperated experiments, we demonstrate (A) whole-body workspace containment, (B) whole-body collision avoidance, and (C) singularity avoidance. In (D), we command a fast sinusoidal end-effector trajectory which, without OSCBF, would lead directly to a collision. The plot visualizes the minimum CBF constraint evolution in this experiment, indicating smooth, safe, and non-overly-conservative behavior from the OSCBF controller.

VI. CONCLUSION

In this work, we have introduced Operational Space Control Barrier Functions as a real-time control method for enforcing safety in hierarchical tasks, in both the operational and joint spaces. By defining the CBF objective in a task-consistent manner, we retain a high level of performance even when the control input is constrained by the safety filter, avoiding both over-conservatism and unnecessary motions. We then scale up the number of CBFs, demonstrating kilohertz control rates even with hundreds of constraints in the QP. This allows for safety even under dynamic motions, or in extremely cluttered environments.

Future work will focus on applications on high-DOF mobile manipulators or bimanual systems, and integration with real-time perception. Moving forward, we aim to apply this controller as a core part of imitation-learning-based manipulation policy training and deployment: collecting data safely via teleoperation, and deploying the learned policies with OSCBF maintaining whole-body safety of the robot.

REFERENCES

- [1] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” *The International Journal of Robotics Research*, 2024.
- [2] A. Khazatsky *et al.*, “Droid: A large-scale in-the-wild robot manipulation dataset,” in *Robotics: Science and Systems*, 2024.
- [3] O. X. E. Collaboration, “Open x-embodiment: Robotic learning datasets and rt-x models,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 6892–6903.
- [4] O. Khatib, “A unified approach for motion and force control of robot manipulators: The operational space formulation,” *IEEE Journal on Robotics and Automation*, vol. 3, pp. 43–53, 1987.
- [5] O. Khatib, “Real-time obstacle avoidance for manipulators and mobile robots,” *The International Journal of Robotics Research*, vol. 5, pp. 90–98, 1986.

- [6] Y. Koren and J. Borenstein, “Potential field methods and their inherent limitations for mobile robot navigation,” in *Proceedings. 1991 IEEE International Conference on Robotics and Automation*, 1991.
- [7] A. Singletary, K. Klingebiel, J. Bourne, A. Browning, P. Tokumaru, and A. Ames, “Comparative analysis of control barrier functions and artificial potential fields for obstacle avoidance,” in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021, pp. 8129–8136.
- [8] O. Khatib, M. Jorda, J. Park, L. Sentis, and S.-Y. Chung, “Constraint-consistent task-oriented whole-body robot formulation: Task, posture, constraints, multiple contacts, and balance,” *The International Journal of Robotics Research*, vol. 41, pp. 1079–1098, 2022.
- [9] S. Kleff, A. Meduri, R. Budhiraja, N. Mansard, and L. Righetti, “High-frequency nonlinear model predictive control of a manipulator,” in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 7330–7336.
- [10] J. Lee, M. Seo, A. Bylard, R. Sun, and L. Sentis, “Real-time model predictive control for industrial manipulators with singularity-tolerant hierarchical task control,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 12 282–12 288.
- [11] A. Singletary, W. Guffey, T. G. Molnar, R. Sinnet, and A. D. Ames, “Safety-critical manipulation for collision-free food preparation,” *IEEE Robotics and Automation Letters*, vol. 7, pp. 10954–10 961, 2022.
- [12] M. A. Murtaza, S. Aguilera, V. Azimi, and S. Hutchinson, “Real-time safety and control of robotic manipulators with torque saturation in operational space,” in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021, pp. 702–708.
- [13] M. Rauscher, M. Kimmel, and S. Hirche, “Constrained robot control using control barrier functions,” in *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2016.
- [14] V. Kurtz, P. M. Wensing, and H. Lin, “Control barrier functions for singularity avoidance in passivity-based manipulator control,” in *2021 60th IEEE Conference on Decision and Control (CDC)*, 2021.
- [15] X. Ding, H. Wang, Y. Ren, Y. Zheng, C. Chen, and J. He, “Online control barrier function construction for safety-critical motion control of manipulators,” *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 54, pp. 4761–4771, 2024.
- [16] W. Shaw Cortez, D. Oetomo, C. Manzie, and P. Choong, “Control barrier functions for mechanical systems: Theory and application to robotic grasping,” *IEEE Transactions on Control Systems Technology*, vol. 29, pp. 530–545, 2021.
- [17] A. Singletary, P. Nilsson, T. Gurriet, and A. D. Ames, “Online active safety for robotic manipulators,” in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019.
- [18] D. Morton, “CBFpy: Control Barrier Functions in Python and Jax.” [Online]. Available: <https://github.com/danielpmorton/cbfpy>
- [19] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, “Control barrier functions: Theory and applications,” in *2019 18th European Control Conference (ECC)*, 2019.
- [20] W. Xiao and C. Belta, “High-order control barrier functions,” *IEEE Transactions on Automatic Control*, vol. 67, 2022.
- [21] J. Breeden and D. Panagou, “High relative degree control barrier functions under input constraints,” in *2021 60th IEEE Conference on Decision and Control (CDC)*, 2021, pp. 6119–6124.
- [22] D. R. Agrawal and D. Panagou, “Safe control synthesis via input constrained control barrier functions,” in *2021 60th IEEE Conference on Decision and Control (CDC)*, 2021, pp. 6113–6118.
- [23] J. Bradbury *et al.*, “JAX: composable transformations of Python + NumPy programs.” [Online]. Available: <http://github.com/google/jax>
- [24] OpenXLA, “A machine learning compiler for gpus, cpus, and ml accelerators.” [Online]. Available: <https://github.com/openxla/xla>
- [25] K. Tracy and Z. Manchester, “On the differentiability of the primal-dual interior-point method,” 2024.
- [26] T. Yoshikawa, “Manipulability of robotic mechanisms,” *The International Journal of Robotics Research*, vol. 4, pp. 3–9, 1985.
- [27] K. Tracy, T. A. Howell, and Z. Manchester, “Differentiable collision detection for a set of convex primitives,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023.
- [28] B. Dai, R. Khorrambakht, P. Krishnamurthy, V. Gonçalves, A. Tzes, and F. Khorrami, “Safe navigation and obstacle avoidance using differentiable optimization based control barrier functions,” *IEEE Robotics and Automation Letters*, vol. 8, pp. 5376–5383, 2023.

VII. APPENDIX

A. Additional Safety Constraints

1) *Velocity Limits*: For a torque-controlled manipulator, joint velocity limits can be enforced as a relative degree 1 CBF, with the following form:

$$h(\mathbf{z}) = \begin{bmatrix} \dot{\mathbf{q}} - \dot{\mathbf{q}}_{\min} \\ \dot{\mathbf{q}}_{\max} - \dot{\mathbf{q}} \end{bmatrix} \quad (49)$$

Similarly, for an operational space velocity or twist limit, we can define

$$h(\mathbf{z}) = \begin{bmatrix} \dot{\nu}(\mathbf{q}, \dot{\mathbf{q}}) - \dot{\nu}_{\min} \\ \dot{\nu}_{\max} - \dot{\nu}(\mathbf{q}, \dot{\mathbf{q}}) \end{bmatrix} \quad (50)$$

where $\dot{\nu}(\mathbf{q}, \dot{\mathbf{q}})$ is defined by the manipulator kinematics, Eq. 7.

For a velocity-controlled manipulator, these must instead be included as a constraint in the QP (see Table I).

2) *Self Collision*: As with the whole-body collision avoidance and set containment CBFs (Eqs. 47, 48), we assume a sphere-based collision model of the robot.

Given a set of self-collision pairs of sphere indices $p_i = (j, k)$, we form the barrier function

$$h_i(\mathbf{z}) = \|\mathbf{x}_{p,j}(\mathbf{q}) - \mathbf{x}_{p,k}(\mathbf{q})\|_2 - r_j - r_k \quad (51)$$

for all i pairs. As before, $\mathbf{x}_p(\mathbf{q})$ represents the position of a sphere after propagation through the forward kinematics, and r is the corresponding radius of the sphere.

Remark: The self-collision model and the whole-body collision avoidance models do not need to have the same geometry. The self-collision model can be a much simpler representation to account for only the possible collision modes, given the robot's kinematics.

3) *Dynamic Obstacles*: For a dynamic obstacle, we can form the following barrier function, again assuming spherized representations of the obstacle (o) and the robot's i^{th} collision body:

$$h_i(\mathbf{z}) = \|\mathbf{x}_{p,i}(\mathbf{q}) - \mathbf{x}_{p,o}\|_2 - \gamma \|\mathbf{v}_{i,\text{rel}}\|_2 - r_i - r_o \quad (52)$$

Here, $\|\mathbf{v}_{i,\text{rel}}\|_2$ represents the norm of the relative velocity between the obstacle and the robot's i^{th} collision body, and γ is a scaling factor. This has the effect of inflating the obstacle by an amount proportional to the relative velocity.

Remark: This is just one of several possible strategies for handling dynamic obstacles that was found to perform well in practice. Here, the scaling factor γ dictates the conservatism of the robot around moving obstacles, similar to how α can be used to tune conservatism. For the previously-mentioned choice of $\alpha = \alpha_2 = 10$, $\gamma = 0.25$ performed well.